

Compte rendu projet programmation répartie

Martin Gouthier
Yanis Husser
Léo Bougnoux
Enzo Poirson

Questions :

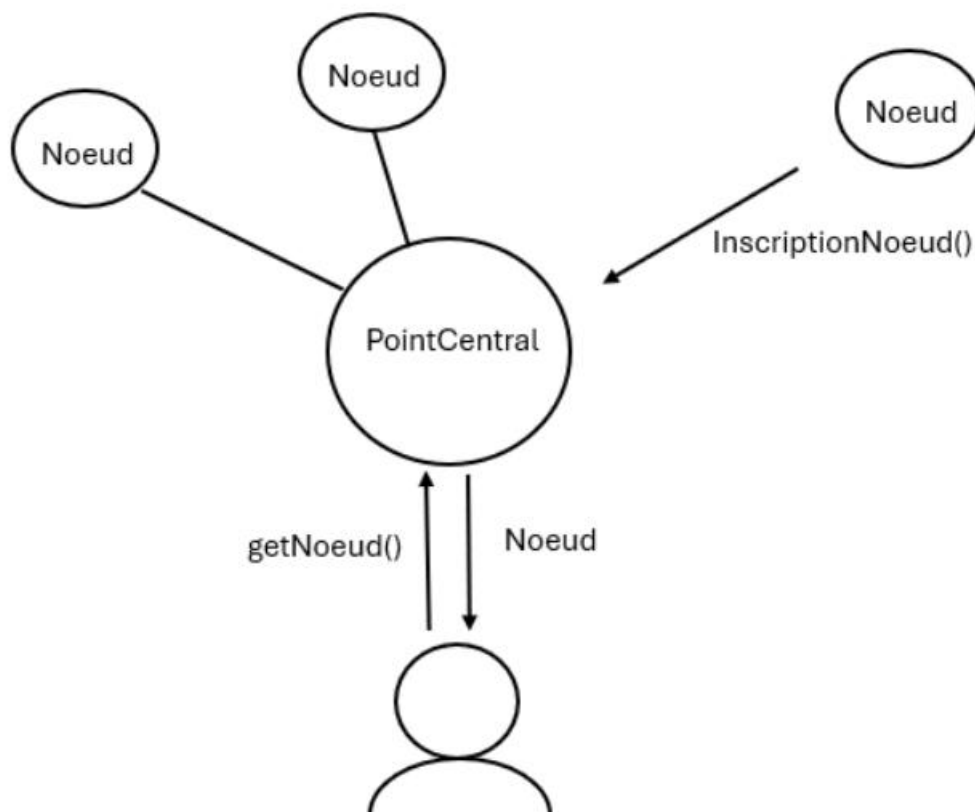


Figure 1 : Schéma de l'inscription d'un nœud et de la récupération d'un nœud par un client

Le point central est initialisé en premier. Il expose une interface permettant à un nœud de s'inscrire ou à un client d'accéder ou de rendre un nœud. Il écoute donc sur un port atteignable grâce à l'annuaire.

Les nœuds sont ensuite initialisés et s'inscrivent au point central. Chaque nœud expose une méthode permettant de générer une partie d'une image globale. Un nœud n'est pas sur écoute d'un port car il n'est pas directement accessible : il faut passer par le point central.

Une fois les nœuds mis en place, le client peut se connecter au point central et demander un nœud. Le point central transmet au client un objet de type `NœudCalcul` capable d'effectuer des calculs.

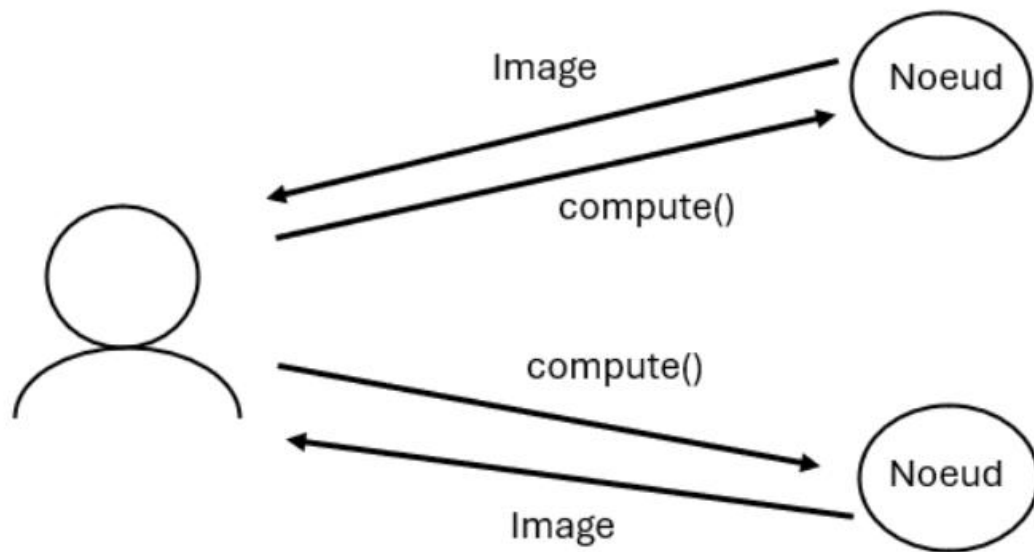


Figure 2 : Schéma de l'utilisation de nœuds par un client.

Le client appelle un certain nombre de nœuds et parallélise le calcul vers chacun de ses nœuds. Le nœud renvoie un objet de type Image. Enfin, le thread utilisé pour paralléliser les appels ajoute la portion d'image à l'image globale, puis se termine. Le client notifie alors le point central que son nœud a terminé et celui-ci est alors de nouveau disponible.

Cas spécifique gérés :

- Dysfonctionnement d'un nœud : En cas d'erreur de retour d'un nœud (RemoteException), le thread contenant le nœud notifie le point central et celui-ci retire le nœud de la liste des nœuds disponibles. Ensuite, le thread redemande un nouveau nœud, puis recrée un nouveau thread qui recommencera l'opération.
- Attente d'un nœud : Si tous les nœuds sont actuellement occupés, le programme va boucler jusqu'à ce qu'un nœud devienne disponible. Si aucun nœud n'est présent alors le programme remonte une exception.
- Gestion dynamique du nombre de nœuds à utiliser : Le programme client prend en paramètre un nombre de nœuds souhaités et détermine le nombre de nœuds réellement utilisés (proche du nombre souhaité, afin de faire un chargement de l'image avec des carrés). Combiné avec le point précédent, cela permet de générer une image composée de beaucoup de sous images même si peu de nœuds sont disponibles.
- Ordonnancement des nœuds à l'aide de la méthode round-robin afin de répartir les requêtes clientes sur le plus de nœuds possibles.

Code des interfaces

Interface du point central :

```
public interface ServiceDistributeurNoeud extends Remote {<
```

```
void inscriptionNoeud(ServiceNoeud noeud) throws RemoteException;
```

```
ServiceNoeud getNoeud() throws RemoteException;
```

```
void libererNoeud(ServiceNoeud noeud) throws RemoteException;
```

```
void suppressionNoeud(ServiceNoeud noeud) throws RemoteException;
```

```
}
```

Interface d'un nœud :

```
public interface ServiceNoeud extends Remote {
```

```
    Image compute(Scene scene, int x, int y, int l, int h) throws RemoteException;
```

```
}
```